Canteen Order, Token, Billing and Preparation Management System

DSA MINI PROJECT — PROJECT SUMMARY

1. Project Overview

The project is a menu-driven C++17 application designed to manage college canteen orders. It generates a unique daily token and bill for each order, prepares an itemised bill, gives priority customers precedence, assigns orders across four serving stations, tracks preparation status, and stores daily records for the manager.

2. Problem Addressed

A college canteen handles many orders at once. Each order needs a unique token, an itemised bill, and an organised service order in which priority customers are served first while multiple orders are prepared in parallel. The system manages these tasks through data structures and file storage.

3. Main Objectives

- Automate order placement with a unique daily token, bill number, and itemised bill.
- Apply a 10% surcharge to priority orders.
- Serve orders using separate priority and normal queues, with priority orders served first.
- Enable token lookup, order tracking, and cancellation.
- Prepare up to four orders simultaneously using four serving stations.
- Store orders and menu prices in text files and generate daily reports, including completed orders sorted by bill amount.

4. Data Structures and Their Uses

Data structure / technique	Purpose in the project
Linked-list FIFO queues	Separate priority and normal queues preserve arrival order within each category. Enqueue and dequeue take O(1) time.
Hash table with separate chaining	Uses 211 buckets and token % 211 to find an order in average O(1) time.
Array of four ServingStation objects	Represents four fixed serving stations, allowing up to four orders to be prepared at the same time.
Vectors	Maintain the order register and menu in insertion order and provide stable pointers to each order.
Linear search and insertion sort	Used for bill-number lookup and sorting completed orders by bill amount.
Sequential text files	Store orders, menu prices, and daily archives, supporting readable storage and restart recovery.

5. Implementation and Tools

The application is implemented in C++17 using an object-oriented design with the classes **MenuItem**, **Order**, **Bill**, **ServingStation**, and **CanteenSystem**. It uses STL containers, **std::thread** with **std::mutex** for the automatic status timer, and **std::filesystem** for daily archives. It is compiled using **g++ -std=c++17 -pthread** as a menu-driven console application.

6. Demonstrated Output

The project report shows the main menu, customer menu with prices and preparation times, order confirmation with an itemised bill, and the daily order register stored in **orders.txt**.

7. Conclusion and Learning

The project demonstrates how choosing suitable data structures improves the canteen workflow: linked queues support fair priority scheduling, a hash table enables fast token lookup, and text files provide persistent records. It also demonstrates combining object-oriented programming with a timer thread and mutex to keep shared data consistent.